Oracle pràctiques: PL/SQL triggers

Consells
El primer trigger està escrit per a SGBD de Oracle.

Tasca 1: El nostre primer trigger.

Analitza el trigger següent:

```
CREATE or REPLACE TRIGGER empleats_dep
BEFORE INSERT OR UPDATE ON empleats
FOR EACH ROW;
DECLARE
        how_many NUMBER(2);
BEGIN
        SELECT COUNT(*) INTO howmany FROM empleats
      WHERE departament = new.departament;
        IF how_many >= 3 THEN
         Raise_application_error(-20000,('There are already 3 ' ||
        :new.departament || ' employes in the departament');
        ENDIF;
END;
```

- **Descriviu amb les teues pròpies paraules el que aquest trigger ha de fer (fent cas omís dels errors que conté!).**

El trigger es dispara quan es fa un update o un insert a la taula empleats, i ho fa abans de fer les modificacions de les dades i per cada una de les files afectades. La ‘‘funció’’ declara la variable ‘‘how_many’’ la qual se li es assignada el nombre de files de la taula empleats que tenen el mateix departament que abans de la modificacio. Si el nombre de files es 3 o més salta un error.

- **Implementa el trigger en la teua BBDD i descriu els errors que té aquest trigger.**

El trigger no pot ser reemplaçat. FOR EACH ROW va sense ; . Des del DECLARE fins a l’END hauria d’anar en una funció apart, la qual retorna TRIGGER. El tipus NUMBER no existeix, hauria de ser INTEGER o NUMERIC. Al SELECT INTO el COUNT(*) va després del INTO, i la variable ‘‘how_many’’ no està escrita correctament. El RAISE_APPLICATION_ERROR(-20000) pot canviar-se per RAISE EXCEPTION, treient les ||, deixant tot el missatge entre cometes simples i utilitzant % per mostrar new.departament. A més, els : davant de new.departament no són correctes. L’ENDIF ha de ser END IF.

Oracle pràctiques: PL/SQL triggers

```
CREATE TRIGGER empleats_dep
BEFORE INSERT OR UPDATE ON empleats
FOR EACH ROW
EXECUTE PROCEDURE emples();

CREATE OR REPLACE FUNCTION emples() RETURNS TRIGGER AS $$
DECLARE
        how_many INTEGER;
BEGIN
        SELECT COUNT(*) INTO how_many FROM empleats
        WHERE departament = new.departament;
        IF how_many >= 3 THEN
        RAISE EXCEPTION 'There are already 3 % employes in the departament',
        new.departament;
        END IF;
END;
$$ LANGUAGE 'plpgsql'
```

Tasca 2: Provant el trigger.

Després de crear qualsevol trigger o procediment, és important provar que funciona correctament.

Comprova les dades que hi ha actualment a la taula d'empleats, crea nous empleats per a un dels departaments.

Has de comprovar que en afegir dos empleats al mateix departament no es dispara el trigger, però el tercer viola la regla de negoci aplicada pel trigger. Així que obtindràs un missatge com el següent:

ERROR at line 1:
ORA-20000: There are already 3 «DEP» employes in the departament
ORA-06512: at "LECT.EMPLEATS_DEP", line 7
ORA-04088: error during execution of trigger 'LECT.EMPLEATS_DEP'

Fes les captures de pantalla pertinets que mostren el procés que has seguit per provar el trigger.

	dni character varying(9)	nom character varying(13)	cognom1 character varying(10)	cognom2 character varying(10)	depa inte
1	07977747J	PEDRO VTE	SÁNCHEZ	MARTÍNEZ	
2	13171339K	LAURA	MATEU	CARBONERES	
3	16283098H	VIRMA	CHENOLL	RUBIO	
4	1931252B	ENCARNACIÓN	BOU	GRAU	

Oracle pràctiques: PL/SQL triggers

```
7   INSERT INTO empleats(dni, nom, cognom1, cognom2, departament, salari) VALUES('HOL
8   DELETE FROM empleats WHERE dni = 'HOLA1'
9
10  CREATE TRIGGER empleats_dep
11  BEFORE INSERT OR UPDATE ON empleats
12  FOR EACH ROW EXECUTE PROCEDURE emples();
13
14  CREATE OR REPLACE FUNCTION emples() RETURNS TRIGGER AS $$
```

```
utput pane
Data Output   Explain   Messages   History

ERROR:  There are already 3 employes in the department 713
********** Error **********

ERROR: There are already 3 employes in the department 713
SQL state: P0001
```

Tasca 3 Ara ja estàs llest per crear el teu propi trigger!

Escenari: Els salaris del personal s'introdueixen, eliminen quan algú se'n va o fins i tot a vegades es van incrementant! La companyia ara requereix que aquests canvis es registren amb la finalitat de crear una pista d'auditoria (no siga que algun empleat expert en BBDD faça un cursor per incrementar-se el sou periodicament).

Crear un trigger que:

- Es dispara cada vegada que s'insereix, s'elimina o s'actualitza un salari en la taula Empleats, (**AFTER**)
- després afegirà un registre a una taula d'auditoria.

Suggeriments:

- Crea la taula d'auditoria en primer lloc, amb els mateixos tipus de dades que en la taula empleats segons correspongui.
- Utilitzeu FOR EACH ROW per assegurar que si s'actualitza més d'un sou, una pista d'auditoria es manté per a cada canvi realitzat (per exemple, és possible que vulgueu augmentar els salaris de tots els empleats per sota del salari mínim)
- SYSDATE es pot utilitzar en qualsevol lloc per obtenir la data actual
- USER es pot utilitzar en qualsevol lloc per obtenir l'usuari actual

Després de crear el trigger amb èxit, proveu-lo a fons mitjançant la realització de les accions apropiades a la taula i la comprovació de les entrades de la taula d'auditoria.

Fes les captures de pantalla pertinents que mostren el procés que has seguit per provar el trigger.

Oracle pràctiques: PL/SQL triggers

```
CREATE OR REPLACE FUNCTION salaris() RETURNS TRIGGER AS $$
DECLARE
num_registre INTEGER;
rec RECORD;
BEGIN
FOR rec IN SELECT * FROM empleats
LOOP
SELECT COUNT(*) INTO num_registre FROM auditoria;
IF (TG_OP = 'DELETE') THEN
INSERT INTO auditoria VALUES((num_registre+1), old.dni, old.nom, old.cognom1,
IF (old.dni NOT IN (SELECT dni FROM empleats)) THEN
RETURN NULL;
END IF;
ELSIF rec.dni = new.dni THEN
IF (TG_OP = 'UPDATE') THEN
```

```
INSERT INTO auditoria VALUES((num_registre+1), rec.dni, rec.nom, rec.cognom1, r
ELSIF (TG_OP = 'INSERT') THEN
INSERT INTO auditoria VALUES((num_registre+1), rec.dni, rec.nom, rec.cognom1, r
END IF;
END IF;
END LOOP;
RETURN NULL;
END;
$$ LANGUAGE 'plpgsql'

CREATE TRIGGER empleats_salari
AFTER INSERT OR UPDATE OR DELETE ON empleats
FOR EACH ROW EXECUTE PROCEDURE salaris();
```

```
SELECT * FROM auditoria

UPDATE empleats SET salari = 10000 WHERE dni = '9814648L'
DELETE FROM empleats WHERE dni = '9814648L'
INSERT INTO empleats VALUES ('9814648L', 'PILAR', 'VALLES', 'CHENOLL', 737, 7256)

CREATE TABLE auditoria(num_registre INTEGER NOT NULL, dni VARCHAR NOT NULL, nom V
DROP TABLE auditoria
```

ane

Output | Explain | Messages | History

num_registre integer	dni character varying	nom character varying	cognom1 character varying	cognom2 character varying	depar integ

Oracle pràctiques: PL/SQL triggers

```
UPDATE empleats SET salari = 10000 WHERE dni = '9814648L'
DELETE FROM empleats WHERE dni = '9814648L'
INSERT INTO empleats VALUES ('9814648L', 'PILAR', 'VALLES', 'CHENOLL', 737, 7256)

CREATE TABLE auditoria(num_registre INTEGER NOT NULL, dni VARCHAR NOT NULL, nom V
DROP TABLE auditoria
```

ne

utput | Explain | Messages | History

cter varying	cognom1 character varying	cognom2 character varying	departament integer	salari_anterior numeric	salari_actual numeric
	VALLES	CHENOLL	737	7256	10000

```
3   SELECT * FROM auditoria
4
5   UPDATE empleats SET salari = 10000 WHERE dni = '9814648L'
6   DELETE FROM empleats WHERE dni = '9814648L'
7   INSERT INTO empleats VALUES ('9814648L', 'PILAR', 'VALLES', 'CHENOLL', 737, 7256)
8
9   CREATE TABLE auditoria(num_registre INTEGER NOT NULL, dni VARCHAR NOT NULL, nom V
10  DROP TABLE auditoria
```

Output pane

Data Output | Explain | Messages | History

	acter varying	cognom1 character varying	cognom2 character varying	departament integer	salari_anterior numeric	salari_actual numeric
1	R	VALLES	CHENOLL	737	7256	10000
2	R	VALLES	CHENOLL	737	10000	0

```
3   SELECT * FROM auditoria
4
5   UPDATE empleats SET salari = 10000 WHERE dni = '9814648L'
6   DELETE FROM empleats WHERE dni = '9814648L'
7   INSERT INTO empleats VALUES ('9814648L', 'PILAR', 'VALLES', 'CHENOLL', 737, 7256)
8
9   CREATE TABLE auditoria(num_registre INTEGER NOT NULL, dni VARCHAR NOT NULL, nom V
10  DROP TABLE auditoria
```

utput pane

Data Output | Explain | Messages | History

	acter varying	cognom1 character varying	cognom2 character varying	departament integer	salari_anterior numeric	salari_actual numeric
1	R	VALLES	CHENOLL	737	7256	10000
2	R	VALLES	CHENOLL	737	10000	0
3	R	VALLES	CHENOLL	737	0	7256

Oracle pràctiques: PL/SQL triggers

Tasca 4: Un altre trigger - integritat referencial

Sabem que un codi de departament de la taula departaments no es pot canviar si hi ha empleats vinculats a la divisió ja que això crearia un problema amb la integritat referencial.

- Crear un disparador que farà els canvis corresponents la taula de personal quan es canvia un codi de departament en la taula departaments.
- Quan es dispara el trigger, AFTER o BEFORE? Explica convenientment el perquè.

R: El trigger ha de ser BEFORE, perquè si no, al haver-hi vinculats empleats al departament, hi haurà errors, mentre que si es manipulen els departaments dels empleats abans, es podrà manipular els codis de departament sense errors.

Després de crear el trigger amb èxit, proveu-lo a fons mitjançant la realització de les accions apropiades a la taula i la comprovació de les dades de la taula empleats.

Fes les captures de pantalla pertinents que mostren el procés que has seguit per provar el trigger.

```
CREATE OR REPLACE FUNCTION deps() RETURNS TRIGGER AS $$
DECLARE
BEGIN
IF (TG_OP = 'DELETE') THEN
UPDATE empleats SET departament = 0 WHERE departament = old.codi;
RETURN old;
ELSIF (TG_OP = 'UPDATE') THEN
UPDATE empleats SET departament = new.codi WHERE departament = old.codi;
RETURN new;
END IF;
END;
$$ LANGUAGE 'plpgsql'
```

```
CREATE TRIGGER dep_empleats
BEFORE UPDATE OR DELETE ON departaments
FOR EACH ROW EXECUTE PROCEDURE deps();
```

```
6
7   UPDATE departaments SET codi = 799 WHERE codi = 702
8   DELETE FROM departaments WHERE codi = 799
9
10  DROP FUNCTION deps();
11  DROP TRIGGER dep_empleats ON departaments
```

utput pane

Data Output | Explain | Messages | History

	codi numeric(4,0)	nom character varying(59)	pressupost integer
35	742	Ciència i Enginyeria Nàutiques	19409
36	743	Matemàtica Aplicada IV	19791
37	744	Enginyeria Telemàtica	5904
38	745	Enginyeria Agroalimentària i Biotecnologia	19216
39	746	Enginyeria de Disseny i Programació de Sistemes Electrònics	14653
40	747	Enginyeria de Serveis i Sistemes d'Informació	11517
41	799	Ciència dels Materials i Enginyeria Metal·lúrgica	6715

Oracle pràctiques: PL/SQL triggers

```
UPDATE departaments SET codi = 799 WHERE codi = 702
DELETE FROM departaments WHERE codi = 799

DROP FUNCTION deps();
DROP TRIGGER dep_empleats ON departaments
```

tput pane

ata Output | Explain | Messages | History

	9)	nom character varying(13)	cognom1 character varying(10)	cognom2 character varying(10)	departament integer	salari integer
56		JORDI	SOLER	CAMPINS	737	9398
57		PILAR	VALLES	CHENOLL	737	7256
58		ISIDRO	CHISVERT	MATEU	738	8876
59		ESTHER	CERVERA	BUENO	739	11107
60		LOURDES	SANZ	SANCHEZ	740	6760
61		ROBERTO	FERRANDO	BOIX	799	12499
62		MARIA ISABEL	PARDO	BALDOVI	799	8470

```
DELETE FROM departaments WHERE codi = 799

DROP FUNCTION deps();
DROP TRIGGER dep_empleats ON departaments
```

utput pane

Data Output | Explain | Messages | History

	codi numeric(4,0)	nom character varying(59)	pressupost integer
34	741	Enginyeria Informàtica	19469
35	742	Ciència i Enginyeria Nàutiques	19469
36	743	Matemàtica Aplicada IV	19791
37	744	Enginyeria Telemàtica	5904
38	745	Enginyeria Agroalimentària i Biotecnologia	19216
39	746	Enginyeria de Disseny i Programació de Sistemes Electrònics	14653
40	747	Enginyeria de Serveis i Sistemes d'Informació	11517

```
DELETE FROM departaments WHERE codi = 799

DROP FUNCTION deps();
DROP TRIGGER dep_empleats ON departaments
```

put pane

ata Output | Explain | Messages | History

	9)	nom character varying(13)	cognom1 character varying(10)	cognom2 character varying(10)	departament integer	salari integer
1		CONSUELO	MIRAVET	BOIX	0	4556
2		ROBERTO	FERRANDO	BOIX	0	12499
3		MARIA ISABEL	PARDO	BALDOVI	0	8470
4		LOURDES	MARTINEZ	SANZ	700	11664
5		LAURA	GIMENO	VERDU	700	10498